leanXMSS candidate (WIP)

Tom Wambsgans

Warning: personal opinion of the author only, subject to change.

A stateful hash-based signature for Ethereum attestations: 2^{32} signatures (1 361 years of 10-second slots), small and fast to verify, and thresholdizable, with a DKG or a trusted dealer. Same hash as leanSPHINCS: $\mathsf{H}(P, tw, M)$ is Keccak-256 truncated to 16 bytes, costs are in Keccak-f[1600] permutations ($\lfloor \ell/136 \rfloor + 1$ for ℓ bytes), and security is NIST level 1.

1 The scheme

- synchronized: each Ethereum slot has its own leaf, so no extra state is needed: double-signing is already a slashable offense
- two layers of XMSS of height 16: each leaf of the top tree signs the root of one bottom tree of 2^{16} slots (7.6 days)
- one-time signature: WOTS^{+}C [1], chains of length 4, and a 128-bit digest read as 64 base-4 digits. Grinding forces the last 6 digits to 0, so those chains are dropped (58 chains left), and the others to sum to $T = 106$: 2^{20} tries. A forger pays the same grinding, so nothing is lost
- every digest is $\mathsf{H}(P, tw, R, M)$, where M is the attestation (bottom layer) or the bottom root (top layer), and the signer grinds the 16-byte randomizer $R = \mathsf{H}(R_0, ctr)$ from $R_0 = \mathsf{H}(S, slot, M)$, for a secret seed S: signing M again gives the same signature
- signature: 2 halves of 1 200 bytes each. The top half is the same for 2^{16} slots, so it can be sent once and cached
- verification: 186 permutations, 93 per layer: 1 for the digest, 68 chain steps, 8 for the leaf (960 bytes) and 16 for the path
- keygen: $16 \cdot 10^6$ permutations for the top tree, and the same for each bottom tree

2 Threshold signing

- t-of-n, with two setups: DKG ceremonies among the operators, with $n = 2f + 1$ and $t = f + 1$ (e.g. 2-of-3, 3-of-5, 4-of-7), or a trusted dealer, with $n = 3f + 1$ and $t = 2f + 1$ as in DVT (e.g. 3-of-4, 5-of-7)
- malicious security against $t - 1$ operators (with abort), at signing and at the ceremonies, assuming the operators first agree, by consensus, on the message to sign
- signing: the same for both setups, no MPC, 944 bytes per signer, 2 rounds

Idea (adapted from PRAWNS [2]). Let F be a threshold PRF: any t operators evaluate it together, $t-1$ learn nothing. A chain of a bottom key starts at $y_0 = F(x_0)$, with $y_{j+1} = \mathsf{H}(y_j)$. The setup computes the chains (in MPC, or the dealer in the clear) and publishes $y_1 \oplus F(x_1)$, $y_2 \oplus F(x_2)$ and y_3, for distinct addresses x_j. To sign position p, the signers reveal $F(x_p)$, which is y_0 or unmasks y_1 or y_2 (nothing for $p = 3$).

F. As in leanSPHINCS: one seed k_T per set T of $t-1$ operators, held by the others, and $F(x) = \bigoplus_T \mathsf{H}(k_T, x)$. When a quorum J signs m, each signer's part of $F(x)$ carries masks $\mathsf{H}(k_{ij}, x, m, J)$, one per co-signer j, which cancel within the signing: parts from different signings don't combine.

2.1 Setup by DKG, $n = 2f+1$, $t = f+1$

Ceremonies at keygen, then for each bottom tree (every 2^{16} slots): they recur, so their honest majority must hold against the same $t-1$ operators as signing, hence $n = 2f+1$ and $t = f+1$. Honest-majority MPC with replicated sharing [3] and a malicious-security check [4], with abort:

- keygen: generate P, the seeds and the pairwise keys k_{ij} (commit-reveal); MPC hashes the chains of the 2^{16} top keys and opens their ends; the top root is the public key. Top keys only sign bottom roots, known at the ceremonies, so they need no masks
- each bottom tree: MPC hashes the chains of its 2^{16} keys (58×3 hashes per key) and opens their ends and masked values; the tree is built in the clear; MPC recomputes the next top key and opens its signature of the bottom root
- public data: 182 MB per bottom tree (masked values and chain ends), kept while the tree is in use. Operators keep only their seeds and pairwise keys

Cost (measured with the leanSPHINCS prototype). $2^{16} \cdot 58 \cdot 3 = 11.4 \cdot 10^6$ MPC hashes per bottom tree, and once at keygen. Traffic sent by the busiest operator (about as much is received):

	traffic	average rate (10-second slots)
2-of-3	51.8 GB	79 KB/s
3-of-5	62.2 GB	95 KB/s
4-of-7	66.6 GB	102 KB/s

2.2 Setup by a trusted dealer, $n = 3f+1$, $t = 2f+1$

The usual DVT setting. For a lifetime of L bottom trees, the dealer, in the clear:

- generates P, one seed k_T per set T of $2f$ operators, and the pairwise keys k_{ij}
- computes the 2^{16} top keys and the top tree, whose root is the public key
- for each of the L bottom trees: computes the chains of its 2^{16} keys from F, publishes their ends and masked values, builds the tree, and signs its root with the next top key
- gives each operator its seeds (those of the sets it is not in) and its pairwise keys, then erases everything

Top keys only sign at setup, so they are never thresholdized. No MPC, so the key is secure against $2f$ operators; consensus on the message tolerates f, as in BLS-based DVT. After L bottom trees, a new setup gives a new public key.

Cost. The dealer hashes the chains and their F values (3 per chain, one hash per seed): about $2 \cdot 10^{10}$ hashes per year of lifetime for 5-of-7, i.e. about 4 minutes on a 16-core laptop. The bottleneck is the public data, 182 MB per bottom tree, i.e. 8.8 GB per year of lifetime with 10-second slots (44 GB for 5 years, 88 GB for 10).

2.3 Signing (both setups, any t, no MPC)

1. agree, by consensus, on (slot, m)
2. each signer sends its part of $F(x_s)$, at an address x_s of the slot: $R_0 = \mathsf{H}(F(x_s), m)$; grind $R = \mathsf{H}(R_0, ctr)$ for the WOTS$^+$C encoding (2^{20} tries, in the clear)
3. each signer sends its parts of F at the selected positions; combine, unmask, and check each value against its chain end

$F(x_s)$ is unpredictable until the slot is signed, and the same for every quorum: signing m again gives the same signature, so a slot never opens two encodings. A secret known to every operator would not do: $t-1$ of them could predict R and search for two attestations with the same encoding (about 2^{64} hashes).

References

[1] A. Hülsing, M. Kudinov, E. Ronen, E. Yogev. *SPHINCS$^+$C: Compressing SPHINCS$^+$ With (Almost) No Cost*. IEEE S&P, 2023. https://eprint.iacr.org/2022/778.

[2] D. Boneh, B. Bünz, J. D., Y. Litchev, K. Nazirkhanova. *PRAWNS: Threshold Hash-Based Signatures from Threshold PRFs*, 2026. https://eprint.iacr.org/2026/2142.

[3] R. Cramer, I. Damgård, Y. Ishai. *Share Conversion, Pseudorandom Secret-Sharing and Applications to Secure Computation*. TCC, 2005. https://doi.org/10.1007/978-3-540-30576-7_19.

[4] V. Goyal, Y. Song. *Malicious Security Comes Free in Honest-Majority MPC*. CRYPTO, 2020. https://eprint.iacr.org/2020/134.